ARM-FM-vs-COMPILER

ARM-FM baseline vs the compiler on MiniGrid DoorKey — side by side

Three independent runs of the four `examples/arm-fm/minigrid-doorkey/tasks.json` tasks, generated by both pipelines. The picture version is `ARM-FM-vs-COMPILER-graphs.md`; the raw Reward Machines, SVGs, and per-run logs live under `docs/artifacts/arm-fm-vs-compiler/`.

Both columns are printed in the paper `REWARD_MACHINE` syntax so the two pipelines read side by side. That is the baseline's native format; the compiler's native output is the numeric `s/i/f/r` format, so its cells below are converted with the project's `compiler_machine_to_paper` adapter. The conversion only renames states to `u0...un` and joins guard literals with `&`; it does not change topology, guards, or rewards, and `REWARD_FUNCTION` keeps every row (including `-> 0`) so nothing the numeric rows carried is lost. The untouched numeric text is preserved in `docs/artifacts/arm-fm-vs-compiler/`.

Setup

Setting	Value
Environment	`examples/arm-fm/minigrid-doorkey/environment.md` (4 propositions: `has_key`, `key_lost`, `door_open`, `at_goal`)
Tasks	the four strings in `tasks.json`, unchanged
Provider / model	`opencode` / `mimo-v2.6-flash` (`.env`, unchanged)
Runs	3 per pipeline, independent (fresh engine, no shared history)
ARM-FM baseline	`rm_generator`, refined by the packaged ARM-FM `rm_critic`, at most 3 attempts
Compiler	proposal -> task critic -> DECLARE/LTLf -> FL-AT/MONA DFA -> RM -> RM critic, at most 3 attempts
Compiler fallback	a task that exhausts its attempts with both critics is re-run with `task_critic=False, rm_critic=False` and tagged in the run log

The two pipelines emit different formats natively (paper `u0...un` vs numeric `s/i/f/r`), so everything below is a semantic comparison, per `docs/decisions/arm-fm/reproduction-boundary.md`.

Headline comparison

Twelve machines per column (4 tasks x 3 runs).

	ARM-FM baseline	**Compiler**
Artifacts produced	12/12, no fallback	12/12, but **9/12 via critics-off fallback** (tasks 1, 3, 4 in all three runs)
Identical across all three runs	3/12 (only task 2)	**12/12**
Terminal reward	1.0 (12/12)	1.1 (12/12)

	ARM-FM baseline	Compiler
Negative penalty transition	6/12	0/12
Absorbing rejecting sink	0/12	9/12
Recovery transition to an earlier state	8/12	0/12

Run outcomes

Pipeline	Task	Run 1	Run 2	Run 3
ARM-FM	1	accepted, 1 attempt	accepted, 1 attempt	accepted, 1 attempt
ARM-FM	2	accepted, 1 attempt	accepted, 1 attempt	accepted, 1 attempt
ARM-FM	3	accepted, **2 attempts**	accepted, 1 attempt	accepted, **3 attempts**
ARM-FM	4	accepted, 1 attempt	accepted, **2 attempts**	accepted, 1 attempt
Compiler	1	accepted, **critics off (fallback)**	accepted, **critics off (fallback)**	accepted, **critics off (fallback)**
Compiler	2	accepted, 1 attempt	accepted, 1 attempt	accepted, 1 attempt
Compiler	3	accepted, **critics off (fallback)**	accepted, **critics off (fallback)**	accepted, **critics off (fallback)**
Compiler	4	accepted, **critics off (fallback)**	accepted, **critics off (fallback)**	accepted, **critics off (fallback)**

Nothing was refused: the compiler's supported clause language covers all four tasks, and every task produced an artifact in all three runs. No ARM-FM run needed its rejected-candidate fallback.

---

1. `Use the key to open the door, then reach the goal.`

The compiler decomposes this into two **hard** clauses, `Precedence(has_key, door_open)` and `Precedence(door_open, at_goal)`, then composes the reachable product.

Run	ARM-FM baseline	Compiler

1	`REWARD_MACHINE:`
	`STATES: u0, u1, u2, u3`
	`INITIAL_STATE: u0`
	`FINAL_STATES: u3`
	`TRANSITION_FUNCTION:`
	`(u0, has_key) -> u1`
	`(u0, else) -> u0`
	`(u1, door_open) -> u2`
	`(u1, key_lost) -> u0`
	`(u1, else) -> u1`
	`(u2, at_goal) -> u3`
	`(u2, else) -> u2`
	`(u3, else) -> u3`
	`REWARD_FUNCTION:`
	`(u0, has_key, u1) -> 0.2`
		`REWARD_MACHINE:`
		`STATES: u0, u1, u2, u3, u4`
		`INITIAL_STATE: u0`
		`FINAL_STATES: u4`
		`DEFAULT_REWARD: 0`
		`TRANSITION_FUNCTION:`
		`(u0, !has_key & !door_open & at_goal) -> u1`
		`(u0, !has_key & door_open & !at_goal) -> u1`
		`(u0, !has_key & door_open & at_goal) -> u1`
		`(u0, has_key & !door_open & !at_goal) -> u2`
		`(u0, has_key & !door_open & at_goal) ->`

Run	ARM-FM baseline	Compiler
	(u1, door_open, u2) -> 0.3 (u2, at_goal, u3) -> 1	u1 (u0, has_key & door_open & !at_goal) -> u1 (u0, has_key & door_open & at_goal) -> u1 (u2, !has_key & !door_open & at_goal) -> u1 (u2, !has_key & door_open & !at_goal) -> u3 (u2, !has_key & door_open & at_goal) -> u1 (u2, has_key & !door_open & at_goal) -> u1 (u2, has_key & door_open & !at_goal) -> u3 (u2, has_key & door_open & at_goal) -> u1 (u3, !has_key & !door_open & at_goal) -> u4 (u3, !has_key & door_open & at_goal) -> u4 (u3, has_key & !door_open & at_goal) -> u4 (u3, has_key & door_open & at_goal) -> u4 REWARD_FUNCTION: (u0, !has_key & !door_open & at_goal, u1) -> 0 (u0, !has_key & door_open & !at_goal, u1) -> 0 (u0, !has_key & door_open & at_goal, u1) -> 0 (u0, has_key & !door_open & !at_goal, u2) -> 0 (u0, has_key & !door_open & at_goal, u1) -> 0 (u0, has_key & door_open & !at_goal, u1) -> 0 (u0, has_key & door_open & at_goal, u1) -> 0 (u2, !has_key & !door_open & at_goal, u1) -> 0 (u2, !has_key & door_open & !at_goal, u3) -> 0.1 (u2, !has_key & door_open & at_goal, u1) -> 0 (u2, has_key & !door_open & at_goal, u1) -> 0 (u2, has_key & door_open & !at_goal, u3) -> 0.1 (u2, has_key & door_open & at_goal, u1) -> 0 (u3, !has_key & !door_open & at_goal, u4) -> 1.1 (u3, !has_key & door_open & at_goal, u4) -> 1.1 (u3, has_key & !door_open & at_goal, u4) -> 1.1

Run	ARM-FM baseline	Compiler
		(u3, has_key & door_open & at_goal, u4) -> 1.1
2	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u3 TRANSITION_FUNCTION: (u0, has_key) -> u1 (u0, else) -> u0 (u1, door_open) -> u2 (u1, key_lost) -> u1 (u1, else) -> u1 (u2, at_goal) -> u3 (u2, else) -> u2 (u3, else) -> u3 REWARD_FUNCTION: (u0, has_key, u1) -> 0.1 (u1, door_open, u2) -> 0.3 (u1, key_lost, u1) -> -0.05 (u2, at_goal, u3) -> 1	REWARD_MACHINE: STATES: u0, u1, u2, u3, u4 INITIAL_STATE: u0 FINAL_STATES: u4 DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, !has_key & !door_open & at_goal) -> u1 (u0, !has_key & door_open & !at_goal) -> u1 (u0, !has_key & door_open & at_goal) -> u1 (u0, has_key & !door_open & !at_goal) -> u2 (u0, has_key & !door_open & at_goal) -> u1 (u0, has_key & door_open & !at_goal) -> u1 (u0, has_key & door_open & at_goal) -> u1 (u2, !has_key & !door_open & at_goal) -> u1 (u2, !has_key & door_open & !at_goal) -> u3 (u2, !has_key & door_open & at_goal) -> u1 (u2, has_key & !door_open & at_goal) -> u1 (u2, has_key & door_open & !at_goal) -> u3 (u2, has_key & door_open & at_goal) -> u1 (u3, !has_key & !door_open & at_goal) -> u4 (u3, !has_key & door_open & at_goal) -> u4 (u3, has_key & !door_open & at_goal) -> u4 (u3, has_key & door_open & at_goal) -> u4 REWARD_FUNCTION: (u0, !has_key & !door_open & at_goal, u1) -> 0 (u0, !has_key & door_open & !at_goal, u1) -> 0 (u0, !has_key & door_open & at_goal, u1) -> 0 (u0, has_key & !door_open & !at_goal, u2) -> 0 (u0, has_key & !door_open & at_goal, u1) -> 0 (u0, has_key & door_open & !at_goal, u1) -> 0 (u0, has_key & door_open & at_goal, u1) -> 0 (u2, !has_key & !door_open & at_goal,

Run	ARM-FM baseline	Compiler
		u1) -> 0 (u2, !has_key & door_open & !at_goal, u3) -> 0.1 (u2, !has_key & door_open & at_goal, u1) -> 0 (u2, has_key & !door_open & at_goal, u1) -> 0 (u2, has_key & door_open & !at_goal, u3) -> 0.1 (u2, has_key & door_open & at_goal, u1) -> 0 (u3, !has_key & !door_open & at_goal, u4) -> 1.1 (u3, !has_key & door_open & at_goal, u4) -> 1.1 (u3, has_key & !door_open & at_goal, u4) -> 1.1 (u3, has_key & door_open & at_goal, u4) -> 1.1
3	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u3 TRANSITION_FUNCTION: (u0, has_key) -> u1 (u0, else) -> u0 (u1, door_open) -> u2 (u1, key_lost) -> u0 (u1, else) -> u1 (u2, at_goal) -> u3 (u2, else) -> u2 (u3, else) -> u3 REWARD_FUNCTION: (u0, has_key, u1) -> 0.1 (u1, door_open, u2) -> 0.3 (u2, at_goal, u3) -> 1	REWARD_MACHINE: STATES: u0, u1, u2, u3, u4 INITIAL_STATE: u0 FINAL_STATES: u4 DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, !has_key & !door_open & at_goal) -> u1 (u0, !has_key & door_open & !at_goal) -> u1 (u0, !has_key & door_open & at_goal) -> u1 (u0, has_key & !door_open & !at_goal) -> u2 (u0, has_key & !door_open & at_goal) -> u1 (u0, has_key & door_open & !at_goal) -> u1 (u0, has_key & door_open & at_goal) -> u1 (u2, !has_key & !door_open & at_goal) -> u1 (u2, !has_key & door_open & !at_goal) -> u3 (u2, !has_key & door_open & at_goal) -> u1 (u2, has_key & !door_open & at_goal) -> u1 (u2, has_key & door_open & !at_goal) -> u3 (u2, has_key & door_open & at_goal) -> u1 (u3, !has_key & !door_open & at_goal) -> u4 (u3, !has_key & door_open & at_goal) -> u4 (u3, has_key & !door_open & at_goal) -> u4 (u3, has_key & door_open & at_goal) ->

Run	ARM-FM baseline	Compiler
		u4 REWARD_FUNCTION: (u0, !has_key & !door_open & at_goal, u1) -> 0 (u0, !has_key & door_open & !at_goal, u1) -> 0 (u0, !has_key & door_open & at_goal, u1) -> 0 (u0, has_key & !door_open & !at_goal, u2) -> 0 (u0, has_key & !door_open & at_goal, u1) -> 0 (u0, has_key & door_open & !at_goal, u1) -> 0 (u0, has_key & door_open & at_goal, u1) -> 0 (u2, !has_key & !door_open & at_goal, u1) -> 0 (u2, !has_key & door_open & !at_goal, u3) -> 0.1 (u2, !has_key & door_open & at_goal, u1) -> 0 (u2, has_key & !door_open & at_goal, u1) -> 0 (u2, has_key & door_open & !at_goal, u3) -> 0.1 (u2, has_key & door_open & at_goal, u1) -> 0 (u3, !has_key & !door_open & at_goal, u4) -> 1.1 (u3, !has_key & door_open & at_goal, u4) -> 1.1 (u3, has_key & !door_open & at_goal, u4) -> 1.1 (u3, has_key & door_open & at_goal, u4) -> 1.1

Obs: both pipelines find the same three-step chain (`has_key` -> `door_open` -> `at_goal`), but they disagree on what a violation is. The baseline recovers or shrugs: key loss either loops in place (`u1 -> u1` , `-0.05` in run 2) or falls back to `u0` for free (runs 1 and 3), and its reward split moves from `0.2/0.3` (run 1) to `0.1/0.3` (runs 2 and 3). The compiler makes the ordering **hard**: every out-of-order valuation drops into state `u1` , an absorbing zero-reward sink with no outgoing rows, and pays `+0.10` on `door_open` plus `+1.10` on completion. The compiler output is byte-identical in all three runs.

---

2. `Eventually acquire the key.`

Run	ARM-FM baseline	Compiler
1	REWARD_MACHINE: STATES: u0, u1 INITIAL_STATE: u0 FINAL_STATES: u1	REWARD_MACHINE: STATES: u0, u1 INITIAL_STATE: u0 FINAL_STATES: u1

Run	ARM-FM baseline	Compiler
	TRANSITION_FUNCTION: (u0, has_key) -> u1 (u0, else) -> u0 (u1, else) -> u1 REWARD_FUNCTION: (u0, has_key, u1) -> 1	DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, has_key) -> u1 REWARD_FUNCTION: (u0, has_key, u1) -> 1.1
2	REWARD_MACHINE: STATES: u0, u1 INITIAL_STATE: u0 FINAL_STATES: u1 TRANSITION_FUNCTION: (u0, has_key) -> u1 (u0, else) -> u0 (u1, else) -> u1 REWARD_FUNCTION: (u0, has_key, u1) -> 1	REWARD_MACHINE: STATES: u0, u1 INITIAL_STATE: u0 FINAL_STATES: u1 DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, has_key) -> u1 REWARD_FUNCTION: (u0, has_key, u1) -> 1.1
3	REWARD_MACHINE: STATES: u0, u1 INITIAL_STATE: u0 FINAL_STATES: u1 TRANSITION_FUNCTION: (u0, has_key) -> u1 (u0, else) -> u0 (u1, else) -> u1 REWARD_FUNCTION: (u0, has_key, u1) -> 1	REWARD_MACHINE: STATES: u0, u1 INITIAL_STATE: u0 FINAL_STATES: u1 DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, has_key) -> u1 REWARD_FUNCTION: (u0, has_key, u1) -> 1.1

Obs: the only task where the four artifacts are structurally and numerically near-identical. Both produce a one-edge machine (`u0` / `0` -> final on `has_key`). The single difference is the reward on the accepting edge: the baseline pays exactly `1` , the compiler pays `1.10` because it folds the clause's `+0.10` completion bonus into the same transition. This is the only compiler task whose RM critic accepted without a fallback.

---

3. `Open the door only after acquiring the key.`

Run	ARM-FM baseline	Compiler
1	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u2 TRANSITION_FUNCTION: (u0, has_key) -> u1 (u0, else) -> u0 (u1, door_open) -> u2 (u1, key_lost) -> u3 (u1, else) -> u1 (u2, else) -> u2 (u3, has_key) -> u1	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u3 DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, !has_key & door_open) -> u1 (u0, has_key & !door_open) -> u2 (u0, has_key & door_open) -> u1 (u2, !has_key & door_open) -> u3 (u2, has_key & door_open) -> u3 REWARD_FUNCTION:

Run	ARM-FM baseline	Compiler
	(u3, else) -> u3 REWARD_FUNCTION: (u0, has_key, u1) -> 0.2 (u1, door_open, u2) -> 1 (u1, key_lost, u3) -> -0.5 (u3, has_key, u1) -> -0.1	(u0, !has_key & door_open, u1) -> 0 (u0, has_key & !door_open, u2) -> 0 (u0, has_key & door_open, u1) -> 0 (u2, !has_key & door_open, u3) -> 1.1 (u2, has_key & door_open, u3) -> 1.1
2	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u2 TRANSITION_FUNCTION: (u0, has_key) -> u1 (u0, else) -> u0 (u1, door_open) -> u2 (u1, key_lost) -> u3 (u1, else) -> u1 (u2, else) -> u2 (u3, has_key) -> u1 (u3, else) -> u3 REWARD_FUNCTION: (u0, has_key, u1) -> 0.3 (u1, door_open, u2) -> 1	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u3 DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, !has_key & door_open) -> u1 (u0, has_key & !door_open) -> u2 (u0, has_key & door_open) -> u1 (u2, !has_key & door_open) -> u3 (u2, has_key & door_open) -> u3 REWARD_FUNCTION: (u0, !has_key & door_open, u1) -> 0 (u0, has_key & !door_open, u2) -> 0 (u0, has_key & door_open, u1) -> 0 (u2, !has_key & door_open, u3) -> 1.1 (u2, has_key & door_open, u3) -> 1.1
3	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u2 TRANSITION_FUNCTION: (u0, door_open & !has_key) -> u3 (u0, has_key) -> u1 (u0, else) -> u0 (u1, door_open) -> u2 (u1, key_lost) -> u0 (u1, else) -> u1 (u2, else) -> u2 (u3, else) -> u3 REWARD_FUNCTION: (u0, door_open & !has_key, u3) - > -1 (u0, has_key, u1) -> 0.2 (u1, door_open, u2) -> 1 (u1, key_lost, u0) -> -0.3	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u3 DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, !has_key & door_open) -> u1 (u0, has_key & !door_open) -> u2 (u0, has_key & door_open) -> u1 (u2, !has_key & door_open) -> u3 (u2, has_key & door_open) -> u3 REWARD_FUNCTION: (u0, !has_key & door_open, u1) -> 0 (u0, has_key & !door_open, u2) -> 0 (u0, has_key & door_open, u1) -> 0 (u2, !has_key & door_open, u3) -> 1.1 (u2, has_key & door_open, u3) -> 1.1

Obs: the clearest divergence, and the only place a baseline run prices the *ordering* itself. Runs 1 and 2 build a four-state machine with a **recovery loop** and key-loss shaping: losing the key drops to `u3` (`-0.5` in run 1) and re-acquiring it costs `-0.1`, while run 2 keeps the same topology but drops both penalties entirely. Run 3 is a third shape again: it adds an explicit `(u0, door_open & !has_key) -> u3` transition paying **`-1`**, the first ARM-FM machine to punish opening the door before acquiring the key, and prices key loss at `-0.3` with a recovery to `u0`. The compiler, by contrast, is identical in all three runs: a three-state product with a rejecting sink where opening the door without the key — or opening it simultaneously with key acquisition — lands in state `u1` and never recovers, and the accepting edge pays `+1.10`. Runs 1, 2, and 3 all required the critics-off fallback.

---

4. `Reach the goal only after opening the door.`

Run	ARM-FM baseline	Compiler

1	`REWARD_MACHINE:`
	`STATES: u0, u1, u2, u3`
	`INITIAL_STATE: u0`
	`FINAL_STATES: u3`
	`TRANSITION_FUNCTION:`
	`(u0, has_key) -> u1`
	`(u0, else) -> u0`
	`(u1, door_open) -> u2`
	`(u1, key_lost & !door_open) -> u0`
	`(u1, else) -> u1`
	`(u2, at_goal) -> u3`
	`(u2, else) -> u2`
	`(u3, else) -> u3`
	`REWARD_FUNCTION:`
	`(u0, has_key, u1) -> 0.1`
	`(u1, door_open, u2) -> 0.3`
	`(u1, key_lost & !door_open, u0) -> -0.1`
	`(u2, at_goal, u3) -> 1`
		`REWARD_MACHINE:`
		`STATES: u0, u1, u2, u3`
		`INITIAL_STATE: u0`
		`FINAL_STATES: u3`
		`DEFAULT_REWARD: 0`
		`TRANSITION_FUNCTION:`
		`(u0, !door_open & at_goal) -> u1`
		`(u0, door_open & !at_goal) -> u2`
		`(u0, door_open & at_goal) -> u1`
		`(u2, !door_open & at_goal) -> u3`
		`(u2, door_open & at_goal) -> u3`
		`REWARD_FUNCTION:`
		`(u0, !door_open & at_goal, u1) -> 0`
		`(u0, door_open & !at_goal, u2) -> 0`
		`(u0, door_open & at_goal, u1) -> 0`
		`(u2, !door_open & at_goal, u3) -> 1.1`
		`(u2, door_open & at_goal, u3) -> 1.1`

2	`REWARD_MACHINE:`
	`STATES: u0, u1, u2, u3`
	`INITIAL_STATE: u0`
	`FINAL_STATES: u3`
	`TRANSITION_FUNCTION:`
	`(u0, has_key) -> u1`
	`(u0, else) -> u0`
	`(u1, key_lost) -> u0`
	`(u1, door_open & has_key) -> u2`
	`(u1, else) -> u1`
	`(u2, at_goal) -> u3`
	`(u2, else) -> u2`
	`(u3, else) -> u3`
	`REWARD_FUNCTION:`
	`(u0, has_key, u1) -> 0.2`
	`(u1, key_lost, u0) -> -0.3`
	`(u1, door_open & has_key, u2) ->`
		`REWARD_MACHINE:`
		`STATES: u0, u1, u2, u3`
		`INITIAL_STATE: u0`
		`FINAL_STATES: u3`
		`DEFAULT_REWARD: 0`
		`TRANSITION_FUNCTION:`
		`(u0, !door_open & at_goal) -> u1`
		`(u0, door_open & !at_goal) -> u2`
		`(u0, door_open & at_goal) -> u1`
		`(u2, !door_open & at_goal) -> u3`
		`(u2, door_open & at_goal) -> u3`
		`REWARD_FUNCTION:`
		`(u0, !door_open & at_goal, u1) -> 0`
		`(u0, door_open & !at_goal, u2) -> 0`
		`(u0, door_open & at_goal, u1) ->`

Run	ARM-FM baseline	Compiler
	0.4 (u2, at_goal, u3) -> 1	0 (u2, !door_open & at_goal, u3) -> 1.1 (u2, door_open & at_goal, u3) -> 1.1
3	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u3 TRANSITION_FUNCTION: (u0, has_key) -> u1 (u0, else) -> u0 (u1, door_open) -> u2 (u1, key_lost & !door_open) -> u0 (u1, else) -> u1 (u2, at_goal) -> u3 (u2, else) -> u2 (u3, else) -> u3 REWARD_FUNCTION: (u0, has_key, u1) -> 0.3 (u1, door_open, u2) -> 0.4 (u1, key_lost & !door_open, u0) -> -0.1 (u2, at_goal, u3) -> 1	REWARD_MACHINE: STATES: u0, u1, u2, u3 INITIAL_STATE: u0 FINAL_STATES: u3 DEFAULT_REWARD: 0 TRANSITION_FUNCTION: (u0, !door_open & at_goal) -> u1 (u0, door_open & !at_goal) -> u2 (u0, door_open & at_goal) -> u1 (u2, !door_open & at_goal) -> u3 (u2, door_open & at_goal) -> u3 REWARD_FUNCTION: (u0, !door_open & at_goal, u1) -> 0 (u0, door_open & !at_goal, u2) -> 0 (u0, door_open & at_goal, u1) -> 0 (u2, !door_open & at_goal, u3) -> 1.1 (u2, door_open & at_goal, u3) -> 1.1

Obs: the task never mentions the key, yet all three baseline runs carry the `has_key` / `door_open` prefix from the DoorKey mission and price key loss (`-0.1` in runs 1 and 3, `-0.3` in run 2); run 2 also tightens the door transition to `door_open & has_key` , while run 3 reaches the same `0.3 / 0.4` shaping without that guard. The compiler ignores `has_key` completely and models only the asked ordering: goal before door (or simultaneous) is an absorbing rejection; goal after door pays `+1.10` . Because the baseline emits no rejecting sink, its out-of-order behaviour is "stay put", not "fail".

Quick tabulation

Task	Run	ARM-FM states	Compiler states	ARM-FM terminal	Compiler terminal	ARM-FM penalties	Compiler penalties	Compiler sink	Identical across all 3 runs
1	1	4	5	1.0	1.1	no	no	yes	—
1	2	4	5	1.0	1.1	yes (-0.05)	no	yes	no / **yes**
1	3	4	5	1.0	1.1	no	no	yes	no / **yes**
2	1	2	2	1.0	1.1	no	no	no	—
2	2	2	2	1.0	1.1	no	no	no	**yes / yes**

Task	Run	ARM-FM states	Compiler states	ARM-FM terminal	Compiler terminal	ARM-FM penalties	Compiler penalties	Compiler sink	Identical across all 3 runs
2	3	2	2	1.0	1.1	no	no	no	**yes / yes**
3	1	4	4	1.0	1.1	yes (-0.5, -0.1)	no	yes	—
3	2	4	4	1.0	1.1	no	no	yes	no / **yes**
3	3	4	4	1.0	1.1	yes (-1.0, -0.3)	no	yes	no / **yes**
4	1	4	4	1.0	1.1	yes (-0.1)	no	yes	—
4	2	4	4	1.0	1.1	yes (-0.3)	no	yes	no / **yes**
4	3	4	4	1.0	1.1	yes (-0.1)	no	yes	no / **yes**

Signal frequency (12 machines per column)

Signal	ARM-FM	Compiler
Terminal reward exactly `1.0`	**12/12**	0/12
Terminal reward `1.1` (reward + clause bonus)	0/12	**12/12**
Negative penalty transition	6/12	0/12
Absorbing rejecting sink	0/12	**9/12**
Recovery transition back to an earlier state	8/12	0/12
Byte-identical across all three runs	3/12	**12/12**
Accepted by its own critics without a fallback	12/12	3/12

The compiler's RM critic rejects its own valid output

The headline result is not the RM shapes — it is that the compiler's RM critic rejected valid machines for tasks 1, 3, and 4 in **all three** runs, forcing the critics-off fallback each time. The rejections are not about reward magnitudes or semantics; they are about the `s:` / `f:` header convention, which the critic prompt itself defines correctly ("`s:` lists non-final numeric states"):

> "Header declares `s: 0,1,2,3` (four non-final states) but rows reference state 4 as a destination and `f: 4`, making state 4 both non-final-listed and final; malformed/inconsistent serialization."
> — RM critic, task 1, attempt 1

> "`f: 4` references a state not listed in `s: 0,1,2,3`; state 4 must be included in the non-final/final declaration consistently."
> — RM critic, task 1, attempt 2

> "the primary defect is `f: 4` referencing a state absent from `s:`, making the serialization malformed."
> — RM critic, task 1, attempt 3

> "Bug: destination state 1 appears in the `s:` list as a non-final state but is never used as a source; ... so opening the door from state 0 loses progress and can never reach final state 3."
> — RM critic, task 4

> "The `s:` list omits state 3, which is declared as the final state in `f:`, making the serialization inconsistent. Additionally, state 0 has a transition on `door_open,at_goal` to state 1 (a non-accepting state) instead of progressing toward acceptance."
> — RM critic, task 4

Both complaints are false positives against the documented format:

- Excluding the final state from `s:` is the serialization rule (`.agents/conventions.md`: "Exclude the final state from the `s` header"). The critic treats a correct header as malformed.
- State `u1` is the **intentional** rejecting sink for a hard precedence violation. The critic reads an absorbing rejection as a reachability bug, and in the task-4 case it misreads the simultaneous `door_open,at_goal` row as "opening the door".

The task critic accepted the same proposals every time, so the failure is isolated to the RM critic stage. Turning the critics off then accepted attempt 1 in every fallback, and those fallback machines are the ones shown above — structurally correct for the requested hard ordering. The practical consequence is that the compiler currently cannot pass its own end-to-end critic gate on any multi-clause precedence task, and the gate only appears to work because the fallback bypasses it.

This is a critic-model limitation, not a prompt omission: the prompt states the convention and the model (`mimo-v2.6-flash`) still misapplies it. Task 2, whose machine has a single row, is the only one that slips through.

Timing

Wall-clock cost per task, measured on the same machine and provider in the same sessions. Compiler figures come from the pipeline's own run logs (`[total …]` records). The ARM-FM pipeline does not log, so its figures are derived from artifact write times; they cover the whole generator + `rm_critic` loop for that task.

	ARM-FM baseline	**Compiler**
Mean per task	**116 s**	**38 s**
Median per task	48 s	38 s
Range per task	20 s – 356 s	10 s – 62 s
Per-run total (4 tasks)	206 s / 256 s / 932 s	34 s / 39 s / 41 s

Compiler detail: one `generate_rm` call with both critics averages **34 s** (n=12); the critics-off fallback averages **5 s** (n=9). The per-task figure above is the critics-on attempt plus its fallback when one was needed.

Obs: the compiler is roughly 3x faster on the mean and much steadier — its per-run totals vary by 20%, against a 4.5x spread for the baseline. The gap is mostly output volume: the compiler asks for a small clause JSON plus two short verdicts, while ARM-FM writes a full paper Reward Machine and a long critique per attempt. ARM-FM's spread is dominated by provider latency variance, not by attempt count: run 3 task 4 cost 316 s in a single attempt, while run 3 task 2 cost 21 s.

Method notes

- The run log records the events of the attempt that produced the artifact. For a fallback task that is the critics-off rerun, so its `run-log.json` shows `task_critic` and `rm_critic` as skipped; the rejected critics-on attempts and their feedback are retained in `logs/run-*.log`.
- Run 2 and run 3 are independent generations, not replays: a fresh engine and no shared history. The compiler's identical output therefore reflects a stable proposal + deterministic compilation, not caching.
- Every artifact is written once and never overwritten; a run directory must be moved aside before re-executing it.